

MCDI500

PROGRAMACIÓN PARA LA CIENCIA DE DATOS

SUMATIVA 2 — FASE 3

Núcleo algorítmico, eficiencia y programación orientada a objetos

Proyecto de mantenimiento predictivo con AI4I 2020

Nombre del estudiante: Alexander Sepúlveda Ormazábal

Grupo: 9

Docente responsable: Omar Salinas Silva

Fecha: 4 de octubre de 2026

Repositorio: github.com/alexander-seor/proyecto-grupo9-mcdi500

Índice

1 Introducción.....	3
2 Diseño de soluciones algorítmicas.....	3
2.1 Codificación funcional y arquitectura del script	3
2.2 Preprocesamiento y transformación del conjunto	4
2.3 Validación técnica y verificación del código	4
2.4 Eficiencia y oportunidades de optimización.....	5
2.5 Diseño estructurado división funcional y escalabilidad	5
3 Código modular y robusto	6
3.1 Programación orientada a objetos.....	6
3.2 Arquitectura funcionalidad y patrón Factory	6
4 Repositorio GitHub	8
5 Notebook ejecutable y productos	8
Conclusiones.....	8
6 Bibliografía y aspectos formales	9
Anexo Matriz de cobertura	10
Anexo Resultados de la verificación final.....	11

1 Introducción

Las fases anteriores definieron el problema y prepararon los datos de AI4I 2020. La Fase 3 reorganiza ese preprocesamiento como un sistema modular, sin cambiar las reglas que ya habían sido justificadas. Esta decisión permite comprobar si una arquitectura orientada a objetos conserva los mismos resultados y, al mismo tiempo, mejora la posibilidad de reutilizar, probar y ampliar el código.

El proyecto estudia condiciones operacionales asociadas con fallas en maquinaria. El monitoreo de condición y los métodos basados en datos ocupan un lugar relevante en el mantenimiento predictivo industrial, pero sus resultados dependen de una preparación reproducible y de una interpretación prudente (Phan et al., 2022). En este trabajo se mantiene la limitación principal: AI4I 2020 es sintético, por lo que las conclusiones describen el conjunto utilizado y no sustituyen una validación con datos obtenidos en maquinaria real.

El objetivo de la fase fue diseñar e implementar un módulo en Python que encapsulara el pipeline de F2 mediante funciones y clases, aplicara herencia, polimorfismo y encapsulamiento, comparara su eficiencia con una versión funcional y dejara evidencia reproducible para la Fase 4. La guía de la asignatura indica que el dato y el pipeline no deben reemplazarse en esta etapa; corresponde reorganizar el mismo proceso y verificar objetivamente su equivalencia (Universidad Andrés Bello, 2026a).

2 Diseño de soluciones algorítmicas

2.1 Codificación funcional y arquitectura del script

La versión funcional se conservó como referencia porque representa de forma directa el flujo desarrollado en F2. Recibe los predictores de entrenamiento y prueba, ajusta OneHotEncoder con Type, ajusta RobustScaler con las cinco variables operacionales y devuelve nueve columnas numéricas. Las entradas y salidas se comparan antes de medir rendimiento, de modo que una diferencia de velocidad no oculte una diferencia de resultado.

La arquitectura modular distribuye las tareas entre carga, transformadores, pipeline, fábrica y medición. Cada función o clase recibe parámetros definidos, trabaja sobre una copia de los datos cuando corresponde y genera errores claros frente a entradas inválidas. La división responde al principio práctico de que cada componente tenga una responsabilidad reconocible.

Componente	Recibe	Devuelve o conserva
cargar_csv	Ruta y nombre descriptivo	DataFrame validado
Transformador.ajustar	Datos de entrenamiento	Parámetros internos y el mismo objeto
Transformador.transformar	DataFrame compatible	Copia transformada
Pipeline.ajustar	Predictores de entrenamiento	Secuencia de pasos ajustados
Pipeline.transformar	Predictores	Nueve columnas procesadas
medir_recursos	Función y repeticiones	Tiempo promedio mínimo y memoria

Tabla 1. Entradas y resultados de los componentes principales

Entrada	Procesamiento	Salida verificada
10.000 × 14	Selección de variables	6 predictores y 1 objetivo
8.000 filas de entrenamiento	Ajuste y transformación	8.000 × 9
2.000 filas de prueba	Transformación sin reajuste	2.000 × 9

Tabla 2. Coherencia entre entradas procesamiento y salidas

2.2 Preprocesamiento y transformación del conjunto

El pipeline conserva Type y cinco mediciones operacionales como predictores. UDI y Product ID se excluyen porque identifican registros; TWF, HDF, PWF, OSF y RNF se excluyen porque describen modos de falla ligados directamente con la etiqueta y podrían introducir fuga de información. Machine failure se mantiene separada durante las transformaciones y se reincorpora al final.

Type se codifica en Type_H, Type_L y Type_M. Las variables de temperatura, velocidad, torque y desgaste se escalan con RobustScaler utilizando parámetros aprendidos solo desde entrenamiento. El ajuste previo a la transformación es relevante porque impide que el conjunto de prueba participe en el cálculo de categorías, medianas o rangos intercuartílicos. Como el diagnóstico de F2 confirmó ausencia de nulos y duplicados, no se agregaron imputaciones ni eliminaciones artificiales.

Grupo	Tratamiento	Justificación
Type	One Hot Encoding	Variable nominal sin orden entre H L y M.
5 variables operacionales	RobustScaler	Conserva valores extremos y reduce su influencia.
Machine failure	Sin transformar	Es la variable objetivo binaria.
2 identificadores	Excluir	No describen condiciones de funcionamiento.
5 modos de falla	Excluir	Evita anticipar la respuesta con variables derivadas de la falla.

Tabla 3. Tratamientos conservados desde la Fase 2

2.3 Validación técnica y verificación del código

La validación cubrió casos normales, límites y excepciones. El caso normal comprobó dimensiones, columnas, ausencia de nulos y equivalencia con F2. Los casos límite utilizaron entradas pequeñas y configuraciones mínimas. Las excepciones provocaron errores esperados al transformar antes de ajustar, solicitar una columna inexistente, entregar una categoría desconocida o indicar un número inválido de repeticiones.

Tipo	Situación probada	Resultado
Normal	Pipeline completo	Dimensiones y esquema correctos.
Normal	Comparación con F2	Resultados equivalentes.
Límite	Entrada mínima	Procesamiento sin perder filas.
Excepción	Transformar antes de ajustar	RuntimeError controlado.
Excepción	Columna inexistente	KeyError controlado.
Excepción	Categoría no aprendida	ValueError controlado.

Tabla 4. Pruebas realizadas en la Fase 3

La comprobación principal se realizó con `pandas.testing.assert_frame_equal`. El conjunto generado por las clases se contrastó con los CSV de F2 usando tolerancias numéricas muy pequeñas. La prueba confirmó que la reorganización no

cambió los datos ni el orden de las columnas. Esta equivalencia es la evidencia más directa de que el cambio de arquitectura no alteró el pipeline.

```
pd.testing.assert_frame_equal(
    entrenamiento_final_f3, entrenamiento_f2,
    check_dtype=False, rtol=1e-12, atol=1e-12,
)
```

Fragmento 1. Comprobación de equivalencia entre F3 y F2

2.4 Eficiencia y oportunidades de optimización

El tiempo se midió con `timeit.repeat` en cinco repeticiones y se informaron el promedio y el mínimo. La documentación de Python recomienda repetir porque otros procesos del equipo pueden afectar una medición breve y señala que el mejor tiempo ayuda a reducir esa interferencia (Python Software Foundation, s. f.-b). La memoria máxima se obtuvo con `tracemalloc`, que registra asignaciones realizadas por Python y permite consultar el pico durante una ejecución (Python Software Foundation, s. f.-c).

Implementación	Promedio s	Mínimo s	Pico MiB
Funcional	0.01327	0.01153	0.663
POO modular	0.03732	0.03517	1.239

Tabla 5. Comparación de tiempo y memoria en la ejecución guardada

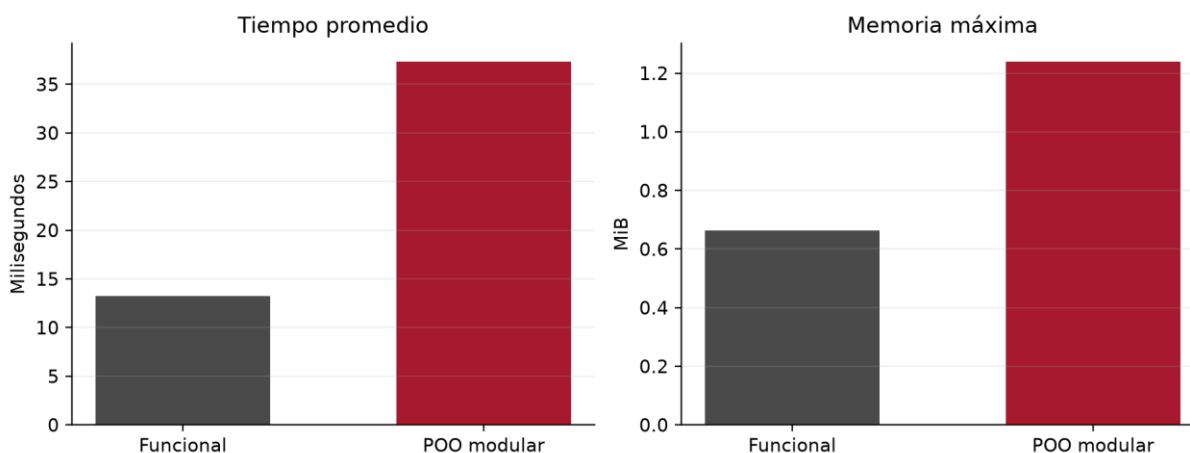


Figura 1. Tiempo promedio y memoria máxima por implementación

La versión funcional registró 0.01327 segundos de tiempo promedio y 0.663 MiB de memoria máxima. La versión orientada a objetos registró 0.03732 segundos y 1.239 MiB. En esta ejecución, la versión modular tardó 2.81 veces más y utilizó 1.87 veces la memoria. El costo se explica por la creación de objetos, las copias intermedias y el escalamiento columna por columna.

La implementación funcional es preferible si el único criterio es el costo de este preprocesamiento pequeño. Para el proyecto se conserva la arquitectura modular porque facilita probar, reemplazar y reutilizar pasos. Una optimización posterior podría agrupar las cinco columnas numéricas en un solo escalador y reducir copias. Las pruebas con 1.000, 5.000, 10.000 y 20.000 filas fueron compatibles con un crecimiento lineal, aunque los tiempos breves y la carga del equipo introducen variación.

2.5 Diseño estructurado división funcional y escalabilidad

La solución utiliza división funcional porque el problema se expresa naturalmente como una secuencia: cargar, seleccionar, codificar, escalar, validar y guardar. No se forzó recursividad, ya que no existe una estructura anidada de profundidad desconocida ni un subproblema que se repita sobre sí mismo. Esta decisión está permitida por la guía, que solicita recursividad o una división funcional bien argumentada (Universidad Andrés Bello, 2026a).

La cohesión es alta porque cada módulo reúne operaciones relacionadas. El acoplamiento se reduce mediante la interfaz ajustar y transformar: Pipeline no necesita conocer el código interno del codificador o del escalador. Para agregar otra transformación basta con implementar la misma interfaz y registrarla en la fábrica.

3 Código modular y robusto

3.1 Programación orientada a objetos

Python permite reunir datos y funcionalidad en clases, mantener estado en las instancias y especializar comportamientos mediante herencia (Python Software Foundation, s. f.-a). Estas capacidades se aplicaron al pipeline porque el codificador y los escaladores deben recordar parámetros aprendidos durante el ajuste para usarlos más tarde en prueba.

El material de la semana presenta la programación orientada a objetos como una forma de construir soluciones escalables y mantenibles mediante clases, atributos y métodos (Universidad Andrés Bello, 2026b). En este caso, una clase sí se justifica porque cada transformador recuerda un estado entre dos momentos: primero aprende con entrenamiento y después aplica exactamente ese aprendizaje a otro conjunto.

Principio	Aplicación en el proyecto	Efecto
Herencia	CodificadorOneHot y EscaladorRobusto heredan de Transformador.	Evita repetir validaciones y control de estado.
Polimorfismo	Pipeline llama ajustar y transformar de igual forma.	Permite recorrer clases distintas con la misma interfaz.
Encapsulamiento	<code>_parametros</code> , <code>_ajustado</code> y <code>_pasos</code> conservan estado interno.	Impide transformar sin ajuste y protege invariantes.

Tabla 6. Principios de programación orientada a objetos aplicados

```
class Transformador:
    def transformar(self, datos):
        if not self._ajustado:
            raise RuntimeError("Ajustar antes de transformar")
        return self.aplicar(datos.copy())
```

Fragmento 2. Control del estado interno en la clase base

El guion bajo comunica que un atributo forma parte de la implementación interna, tal como establece la convención de Python para miembros no públicos (Python Software Foundation, s. f.-a). El encapsulamiento se complementa con verificaciones: el objeto rechaza la transformación si todavía no ha aprendido sus parámetros.

3.2 Arquitectura funcionalidad y patrón Factory

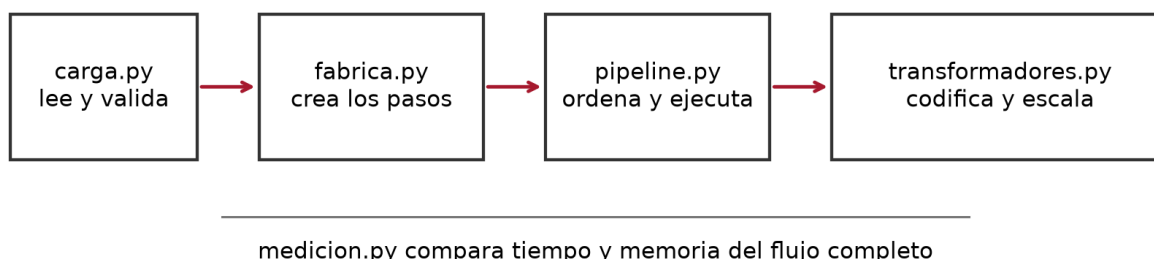


Figura 2. Relación entre los módulos principales de F3

Se aplicó Factory porque el pipeline necesita crear objetos distintos a partir del rol de cada variable. FabricaTransformadores devuelve CodificadorOneHot para una variable categórica y EscaladorRobusto para una variable numérica. Sin esta clase, la selección del objeto quedaría repetida en el notebook o dentro del pipeline. La fábrica concentra esa decisión y permite extenderla en un solo punto.

La evolución respecto de F2 fue gradual. En F2, la codificación y el escalamiento estaban encadenados como funciones dentro del notebook. En F3, las mismas reglas se trasladaron a clases con estado, el orden se delegó a Pipeline y la construcción de los pasos se concentró en FabricaTransformadores. La comparación con los archivos anteriores permitió hacer este cambio sin suponer que el nuevo diseño era correcto solo porque ejecutaba sin errores.

No se incorporaron patrones adicionales porque no resolvían una necesidad concreta del proyecto. La elección sigue la recomendación de la guía de aplicar un patrón cuando corresponde y explicar el problema que resuelve (Universidad Andrés Bello, 2026a).

Módulo	Responsabilidad	Motivo de separación
carga.py	Localizar y validar archivos	Aísla la entrada de datos.
transformadores.py	Codificar y escalar	Agrupar clases con la misma interfaz.
pipeline.py	Ordenar y ejecutar pasos	Separa coordinación de transformación.
fabrica.py	Crear transformadores	Centraliza la elección de clases.
medicion.py	Medir tiempo y memoria	Evita mezclar rendimiento con negocio.

Tabla 7. Arquitectura modular y responsabilidades

4 Repositorio GitHub

El repositorio conserva F1, F2 y F3 bajo una misma raíz. Las rutas del notebook se resuelven desde esa estructura y no contienen carpetas personales. .gitignore excluye el entorno virtual, cachés y notas personales; requirements.txt registra las dependencias; los README de la raíz y de F3 explican el orden de ejecución y los productos generados.

```
F3/
├── S2_F3_NucleoAlgoritmico_Eficiencia_POO.ipynb
├── README.md
├── src/ carga.py · transformadores.py · pipeline.py · fabrica.py · medicion.py
├── data/processed/ conjuntos · diccionario · parámetros
└── docs/ arquitectura · eficiencia · crecimiento · metadatos
```

Fragmento 3. Estructura principal de la Fase 3

La evolución quedó registrada en commits descriptivos. El commit 7ee5001 incorporó la base orientada a objetos y el commit 858ba05 modularizó el pipeline y agregó la evaluación de eficiencia. El trabajo corresponde a un solo estudiante, por autorización del docente, por lo que no se atribuyen aportes a integrantes inexistentes.

5 Notebook ejecutable y productos

S2_F3_NucleoAlgoritmico_Eficiencia_POO.ipynb integra la explicación y la evidencia ejecutable. Se verificaron el intérprete del entorno virtual, las versiones, la semilla 42 y las rutas relativas. La última ejecución completó 31 celdas de código en orden continuo y sin salidas de error. Cada bloque técnico está precedido por una celda que explica para qué se realiza.

El avance formativo aportó la versión funcional y la primera comparación de recursos. La entrega sumativa incorporó la clase base, las clases hijas, el pipeline polimórfico, el patrón Factory, las pruebas de equivalencia y los cuatro artefactos que recibe F4. Esta secuencia queda visible en las secciones del notebook y en el historial de Git.

Producto	Ubicación	Uso posterior
Entrenamiento procesado	F3/data/processed	Análisis de F4
Prueba procesada	F3/data/processed	Comprobación independiente
Diccionario final	F3/data/processed	Interpretar las columnas
Parámetros aprendidos	F3/data/processed	Trazabilidad del ajuste
Comparación de eficiencia	F3/docs	Justificar la implementación
Arquitectura y metadatos	F3/docs	Reproducir y auditar la fase

Tabla 8. Productos persistidos para la entrega y la Fase 4

Los CSV se leen nuevamente después de guardarlos para verificar dimensiones, columnas y ausencia de nulos nuevos. Los cuatro artefactos de transición permiten que F4 se concentre en explorar, visualizar y comunicar. Volver a codificar o escalar los datos finales duplicaría transformaciones y cambiaría su significado.

Conclusiones

La Fase 3 convirtió el pipeline de F2 en una arquitectura formada por módulos y clases. La equivalencia formal entre ambas fases confirmó que la reorganización mantuvo los resultados: 8.000 filas de entrenamiento y 2.000 de prueba, nueve columnas numéricas, sin valores faltantes y con la variable objetivo conservada.

La versión funcional presentó el menor costo de ejecución. La versión orientada a objetos agregó tiempo y memoria, pero hizo explícitas las responsabilidades, el estado y los controles. En este proyecto, la decisión se justifica por mantenibilidad y reutilización. El principal ajuste futuro consiste en escalar juntas las variables numéricas para disminuir copias y llamadas repetidas.

El patrón Factory resolvió una necesidad concreta: crear el transformador según el rol de la variable sin incorporar decisiones de construcción dentro del pipeline. La división funcional resultó más pertinente que una recursión artificial. Los archivos finales dejan a F4 una entrada documentada y evitan repetir la limpieza o el escalamiento.

6 Bibliografía y aspectos formales

Phan, T. L. J., Gehrhardt, I., Heik, D., Bahrpeyma, F., & Reichelt, D. (2022). A systematic mapping study on machine learning techniques applied for condition monitoring and predictive maintenance in the manufacturing sector. *Logistics*, 6(2), 35. <https://doi.org/10.3390/logistics6020035>

Python Software Foundation. (s. f.-a). *Classes*. Python 3.13 documentation. Recuperado el 4 de octubre de 2026, de <https://docs.python.org/3.13/tutorial/classes.html>

Python Software Foundation. (s. f.-b). *timeit* — Measure execution time of small code snippets. Python 3.13 documentation. Recuperado el 4 de octubre de 2026, de <https://docs.python.org/3.13/library/timeit.html>

Python Software Foundation. (s. f.-c). *tracemalloc* — Trace memory allocations. Python 3.13 documentation. Recuperado el 4 de octubre de 2026, de <https://docs.python.org/3.13/library/tracemalloc.html>

Universidad Andrés Bello. (2026a). *Guía de apoyo Sumativa 2 Fase 3 [Documento de apoyo docente]*. MCDI500 Programación para la Ciencia de Datos.

Universidad Andrés Bello. (2026b). *MCDI500 Fase 3 Semana 2 [Material de clase]*. MCDI500 Programación para la Ciencia de Datos.

Anexo Matriz de cobertura

Criterio	Evidencia	Estado
Portada e índice	Portada institucional y campo de índice	Cumplido
Codificación funcional	Sección 2.1 y notebook	Cumplido
Preprocesamiento	Sección 2.2 y equivalencia con F2	Cumplido
Validación	Sección 2.3 casos normales límite y excepciones	Cumplido
Eficiencia	Sección 2.4 tiempo memoria y crecimiento	Cumplido
Diseño estructurado	Sección 2.5 cohesión acoplamiento y división	Cumplido
POO	Sección 3.1 herencia polimorfismo y encapsulamiento	Cumplido
Arquitectura	Sección 3.2 patrón Factory y módulos	Cumplido
Repositorio	Sección 4 README módulos y commits	Cumplido
Notebook	Sección 5 ejecución completa y productos	Cumplido
Bibliografía	Seis fuentes citadas y listadas	Cumplido

Tabla A1. Cobertura de los criterios de la Sumativa 2

Anexo Resultados de la verificación final

El cierre del notebook vuelve a leer los archivos persistidos y comprueba que el traspaso a F4 conserve la estructura validada. La tabla resume las salidas visibles de la última ejecución completa.

Control	Resultado observado	Estado
Ejecución del notebook	31 celdas de código consecutivas y 0 errores	OK
Equivalencia con F2	Entrenamiento y prueba coinciden	OK
Conservación de filas	10.000 de 10.000 registros	OK
Esquema	9 columnas en entrenamiento y prueba	OK
Valores faltantes nuevos	0	OK
Variable objetivo	Machine failure presente y completa	OK
Diccionario final	9 variables documentadas	OK
Parámetros	6 pasos documentados	OK
Persistencia	9 archivos guardados y verificados	OK

Tabla B1. Evidencia del cierre reproducible de la Fase 3